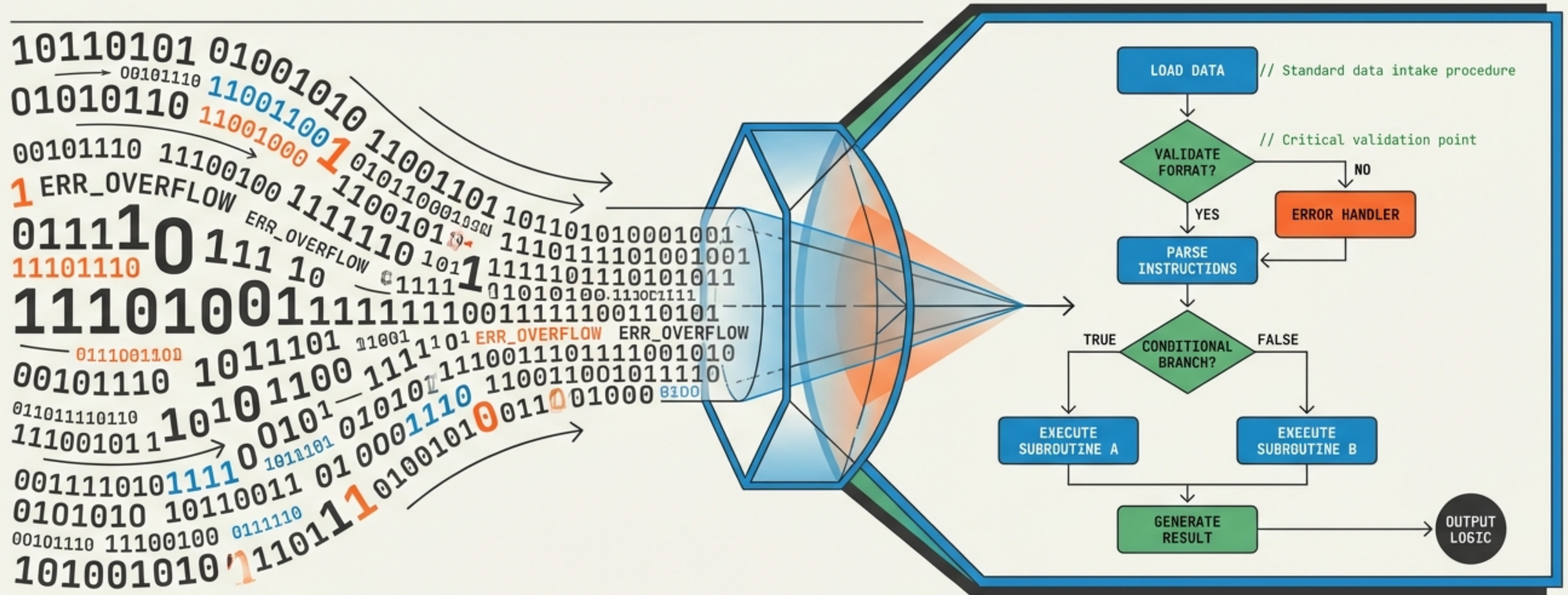


Code Constructs & Binary Analysis

From Raw Assembly to Programmer Intent



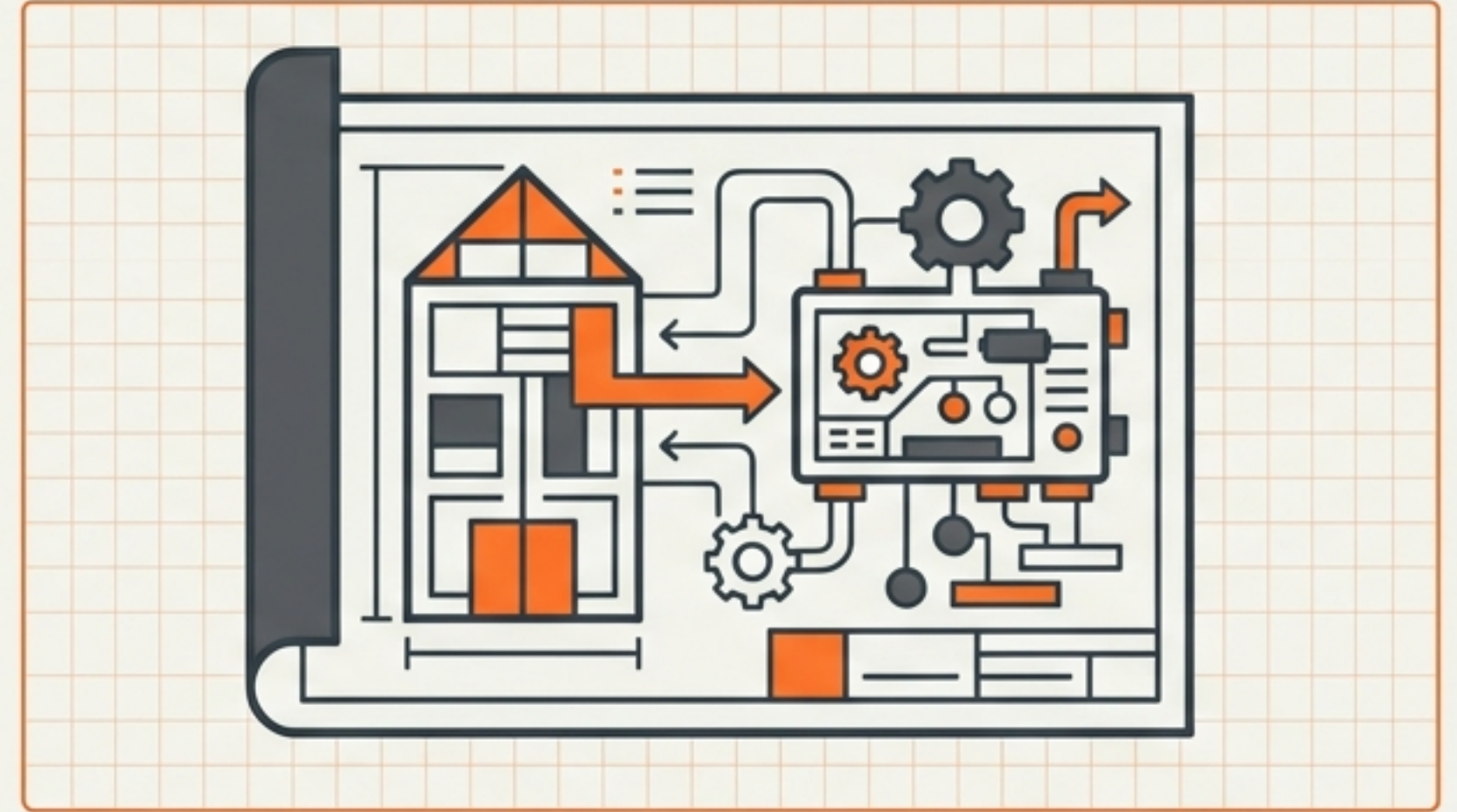
The Analyst's Mission: Reconstructing Logic

Micro View: Instruction Level



Necessary for syntax, but easy to lose context.

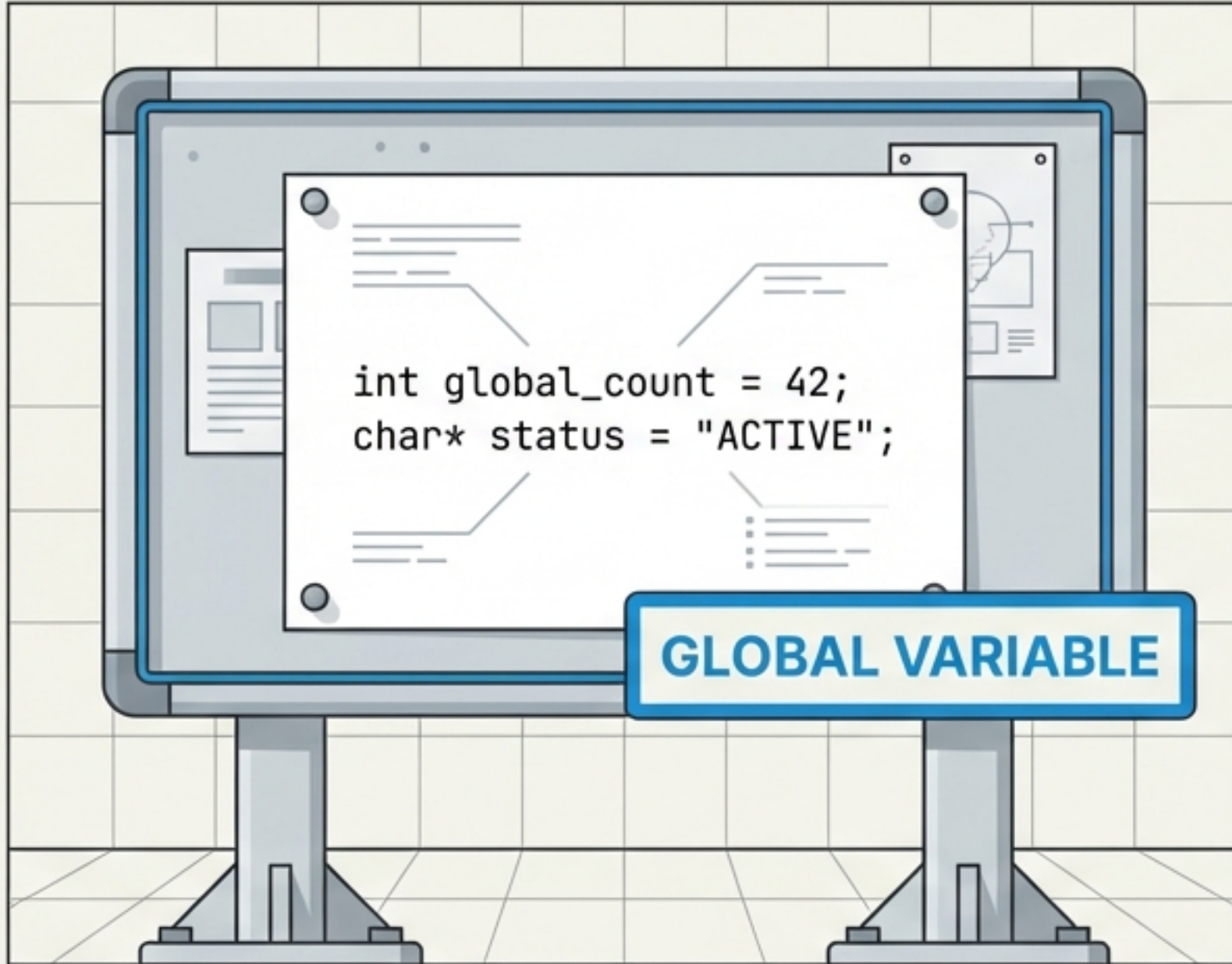
Macro View: Functional Level



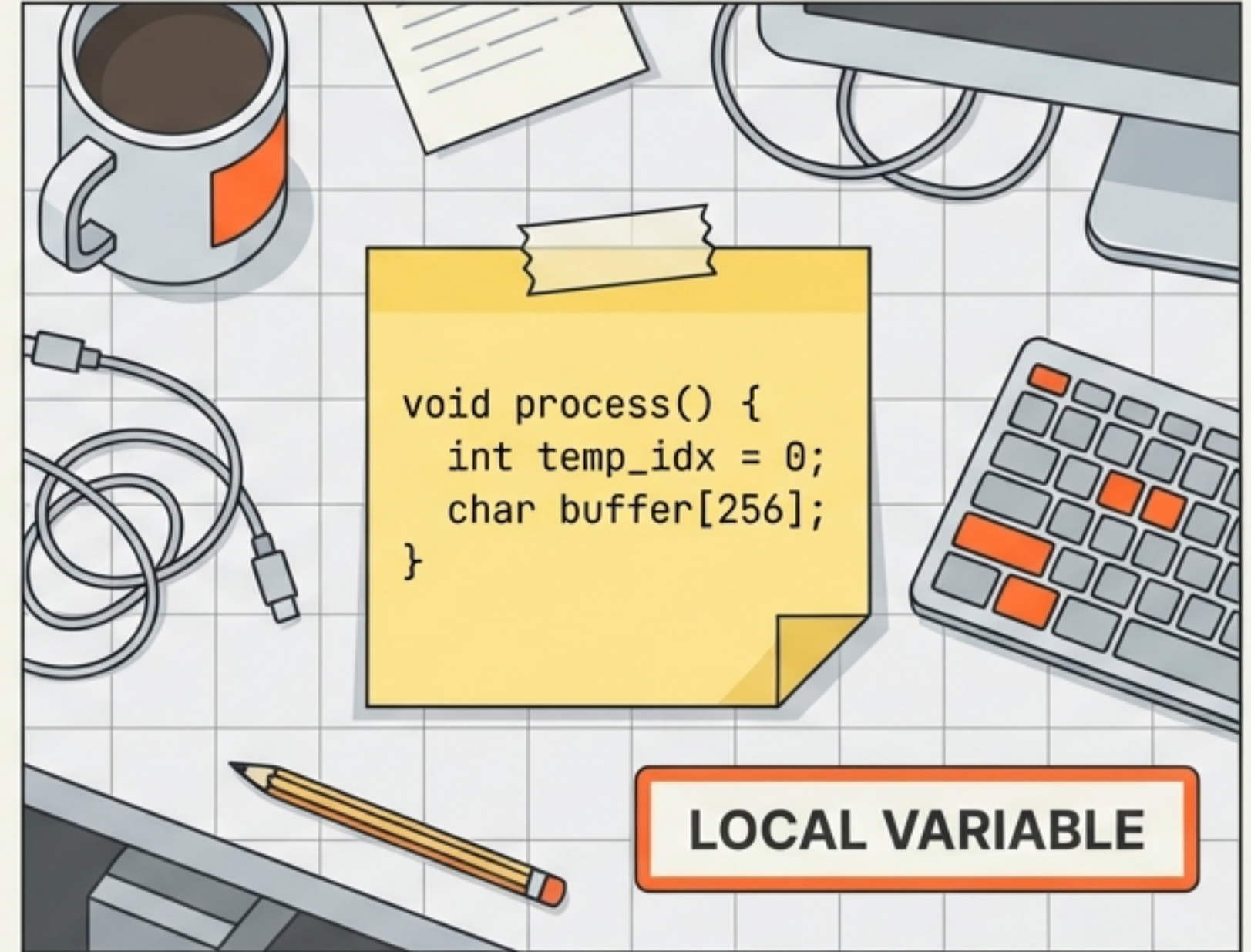
The Goal: Identify behavior (e.g., '**Opens Socket**', 'Edits Registry').

We lack source code. We must reverse the process to find the story hidden in the instructions.

The Landscape of Data: Where Variables Live



- Fixed Location (.data/.bss)
- Visible to All
- Permanent



- Temporary Location (Stack)
- Context Dependent
- Relative to You (EBP)

Analyzing Global Variables

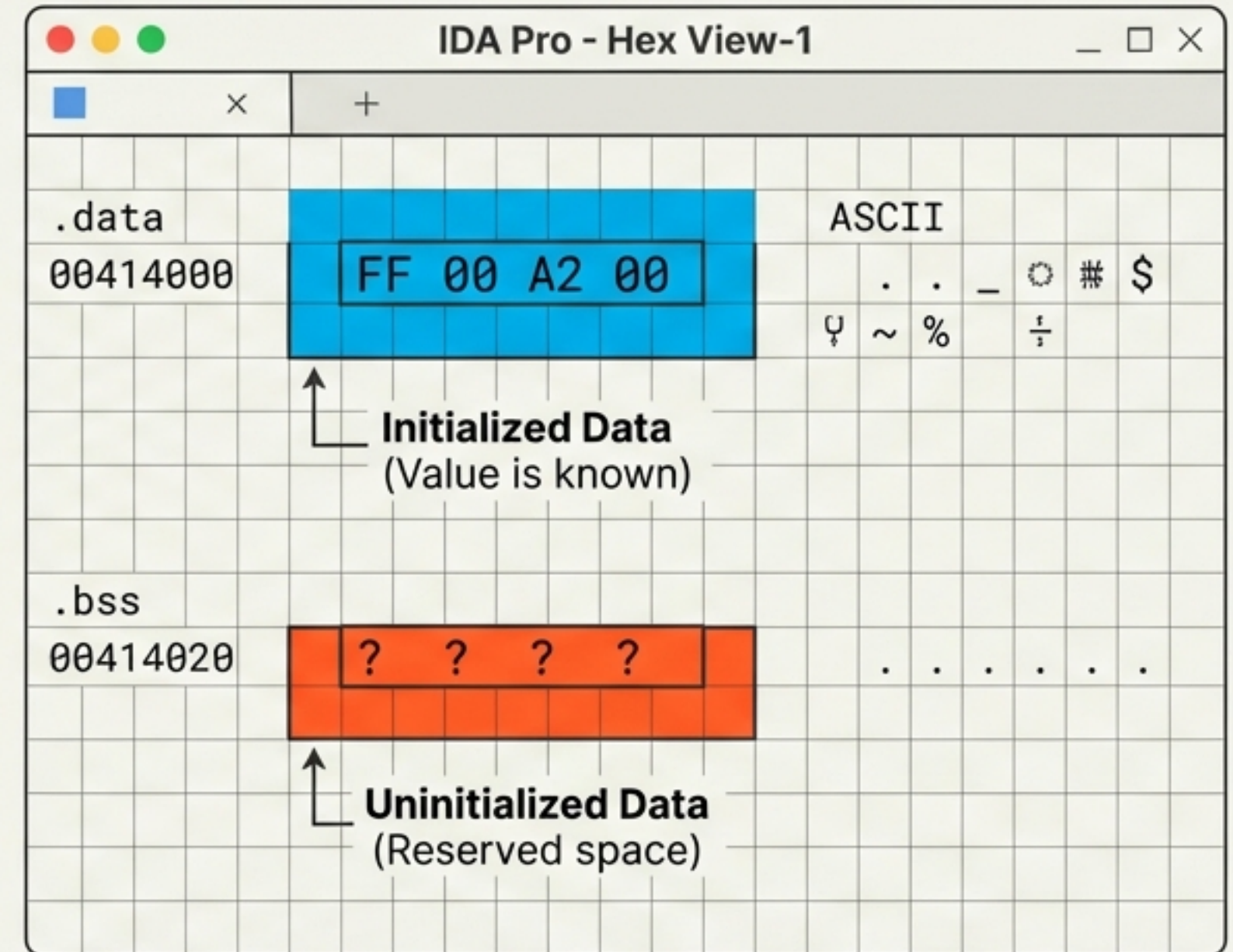
Definition: Accessible program-wide.

Location: .data (Initialized) or .bss (Uninitialized).

Identification: Look for hardcoded memory addresses.

Example Code:

```
mov eax, dword_414000
```



Analyzing Local Variables

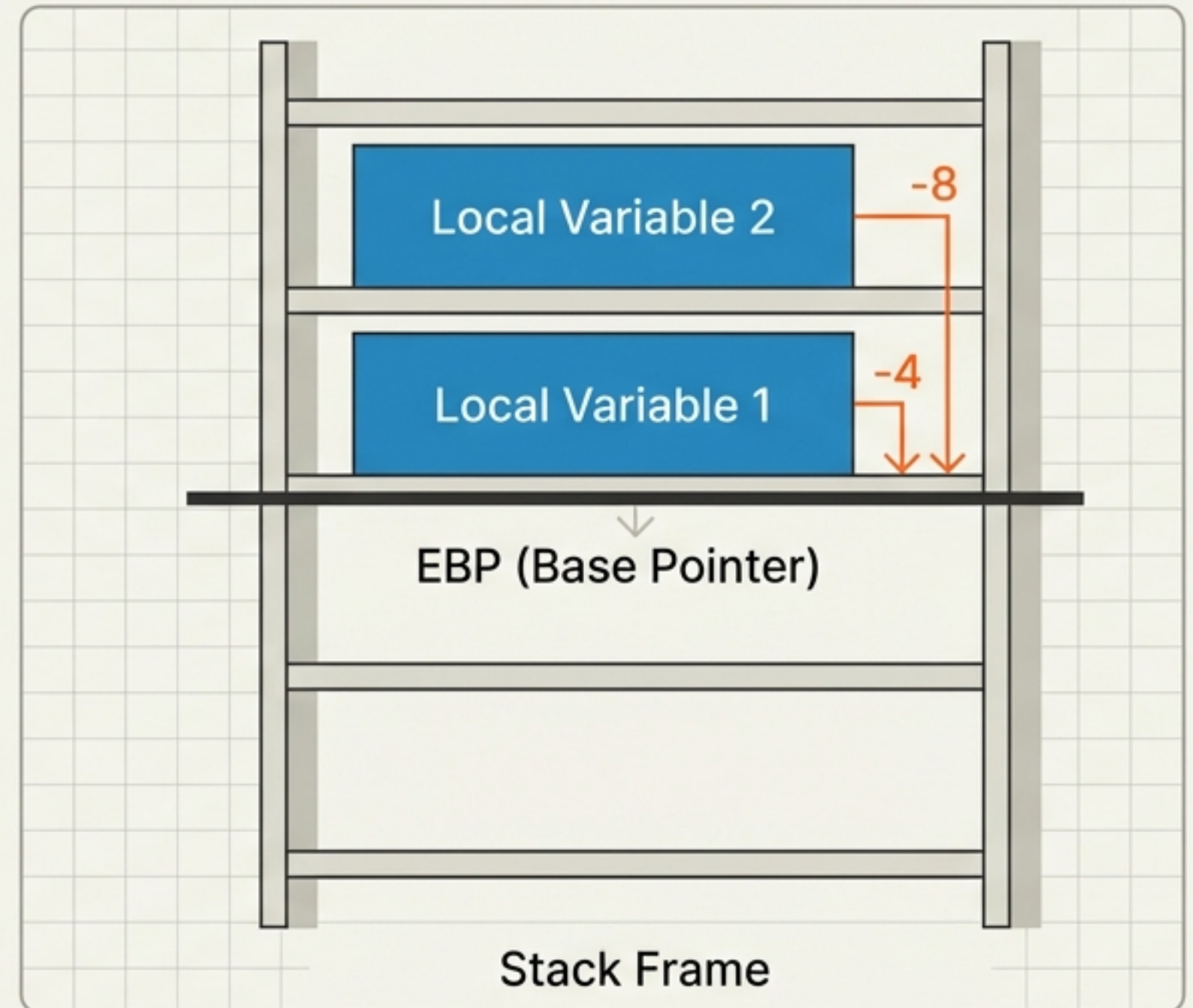
Definition: Accessible only within a function.

Location: The Stack (and sometimes registers).

Identification: Look for offsets relative to pointers.

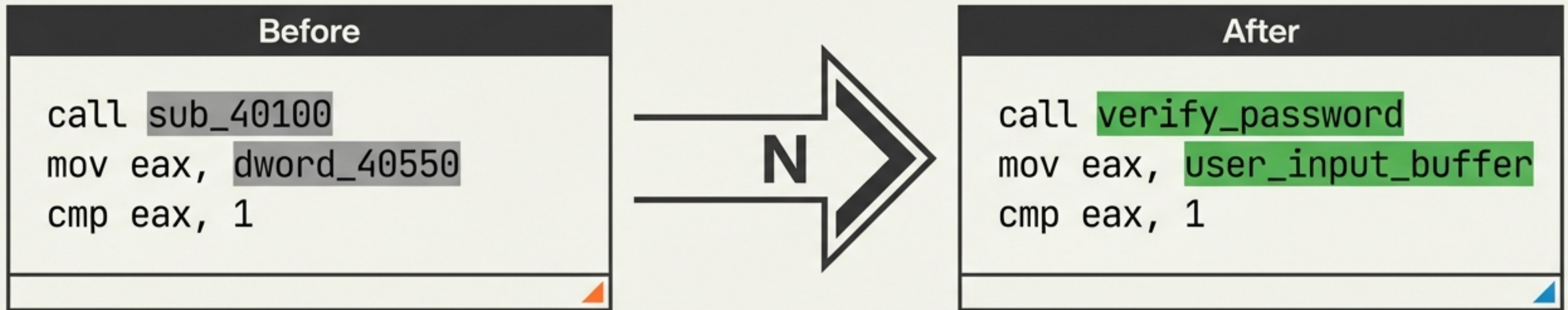
Base Pointer (EBP): The anchor. Variables are at negative offsets (e.g., $ebp - 4$).

Stack Pointer (ESP): The top. Variables are at positive offsets (e.g., $esp + 8$).



Renaming is Analysis

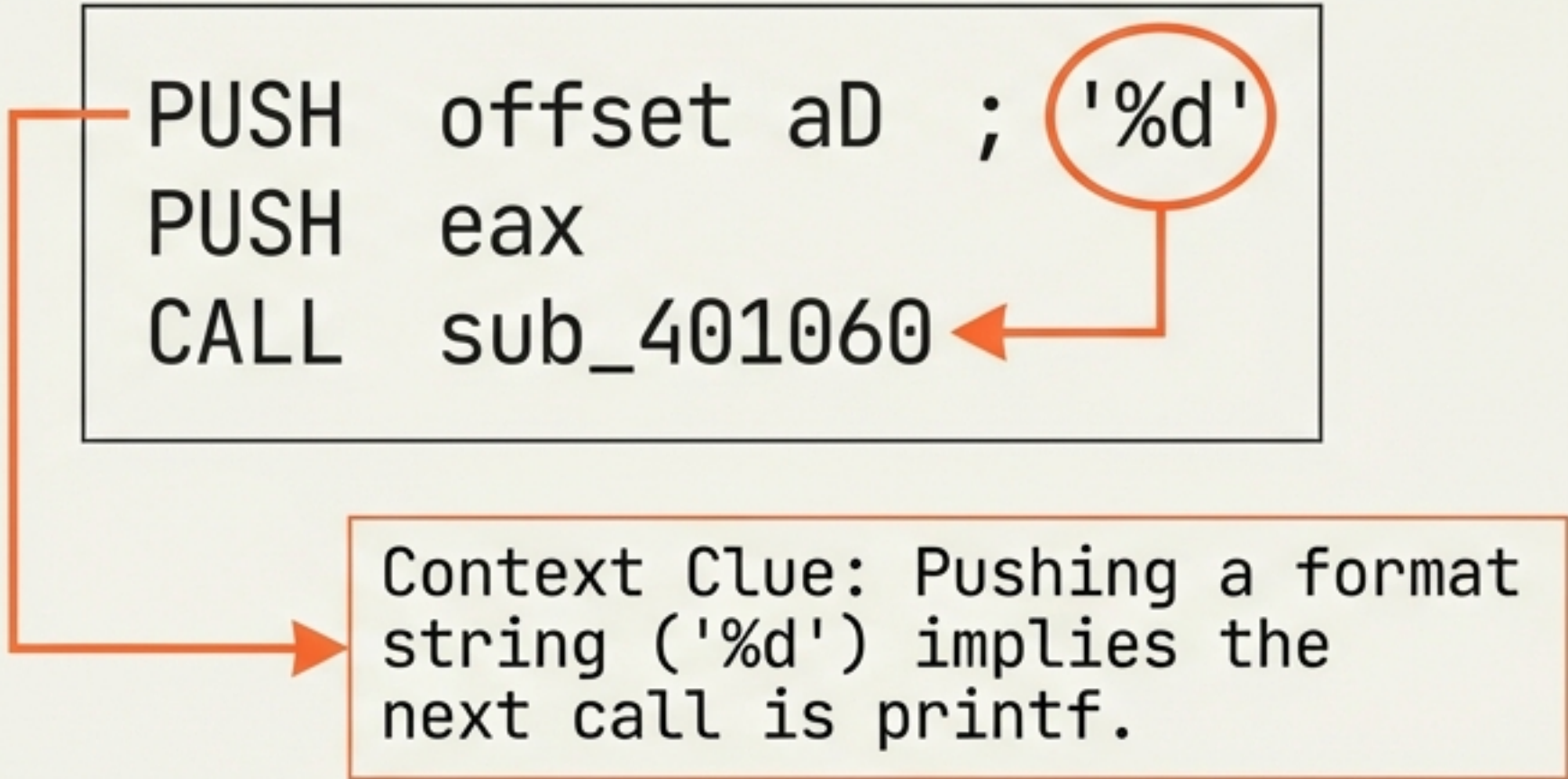
The most powerful tool: Hotkey “N”



An unnamed variable is an unanalyzed variable. Renaming propagates meaning through the entire database.

Identifying Standard Library Functions

Don't analyze standard code (like printf).
Identify it by context and move on.



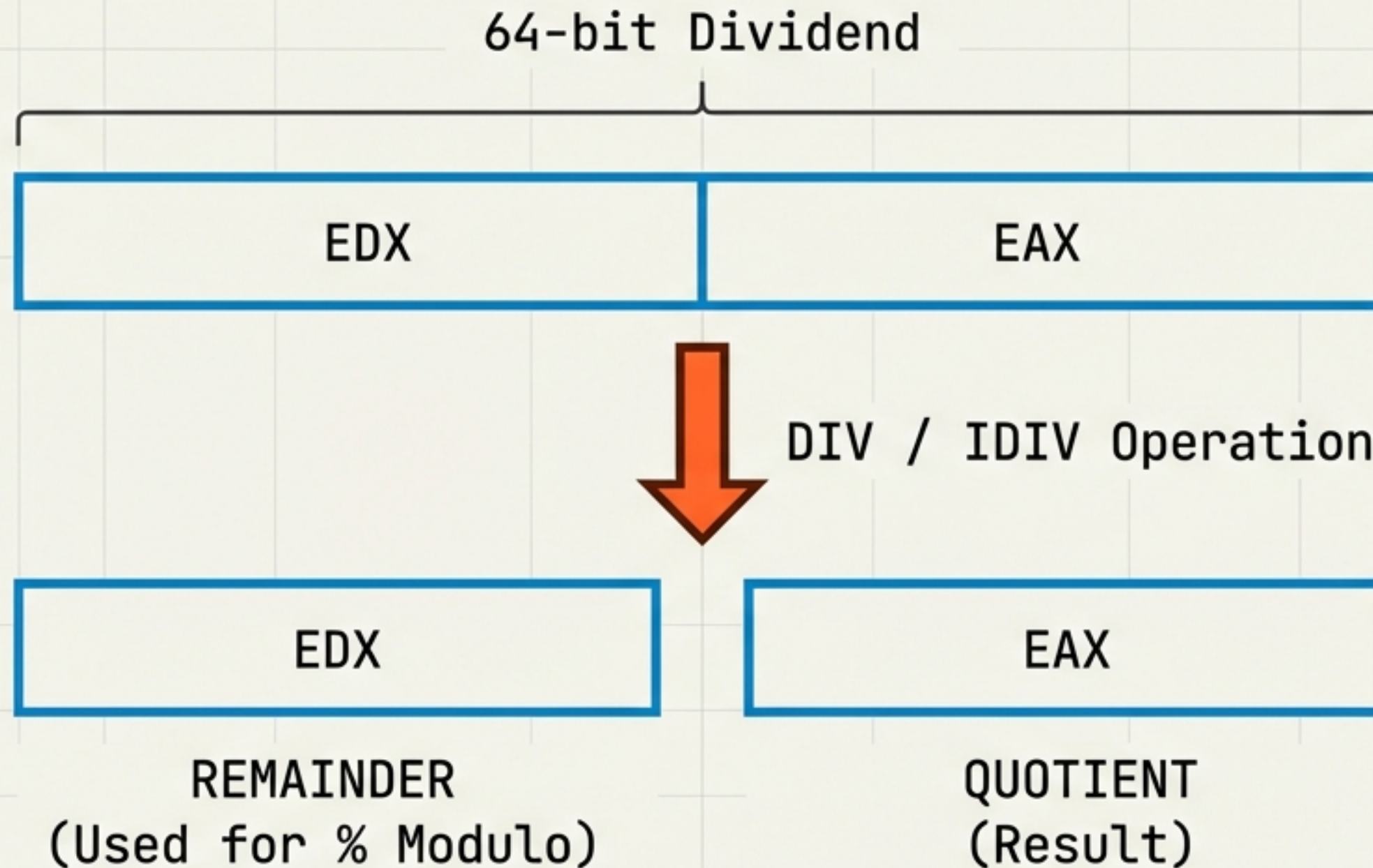
```
PUSH  offset aD ; '%d'  
PUSH  eax  
CALL  sub_401060
```

The assembly code is enclosed in a black rectangular box. The string `'%d'` in the first line is circled in orange. An orange arrow points from this circle to a text box below. Another orange arrow points from the left side of the code box to the same text box.

Context Clue: Pushing a format string ('%d') implies the next call is printf.

Arithmetic Mechanics: Division

32-bit division uses a 64-bit pair. It is not contained in a single register.



The “CDQ” Gotcha



CDQ

CDQ (Convert Double to Quad): Sign-extends EAX into EDX.

EAX

-1 11111111111111111111111111111111

CDQ

EDX

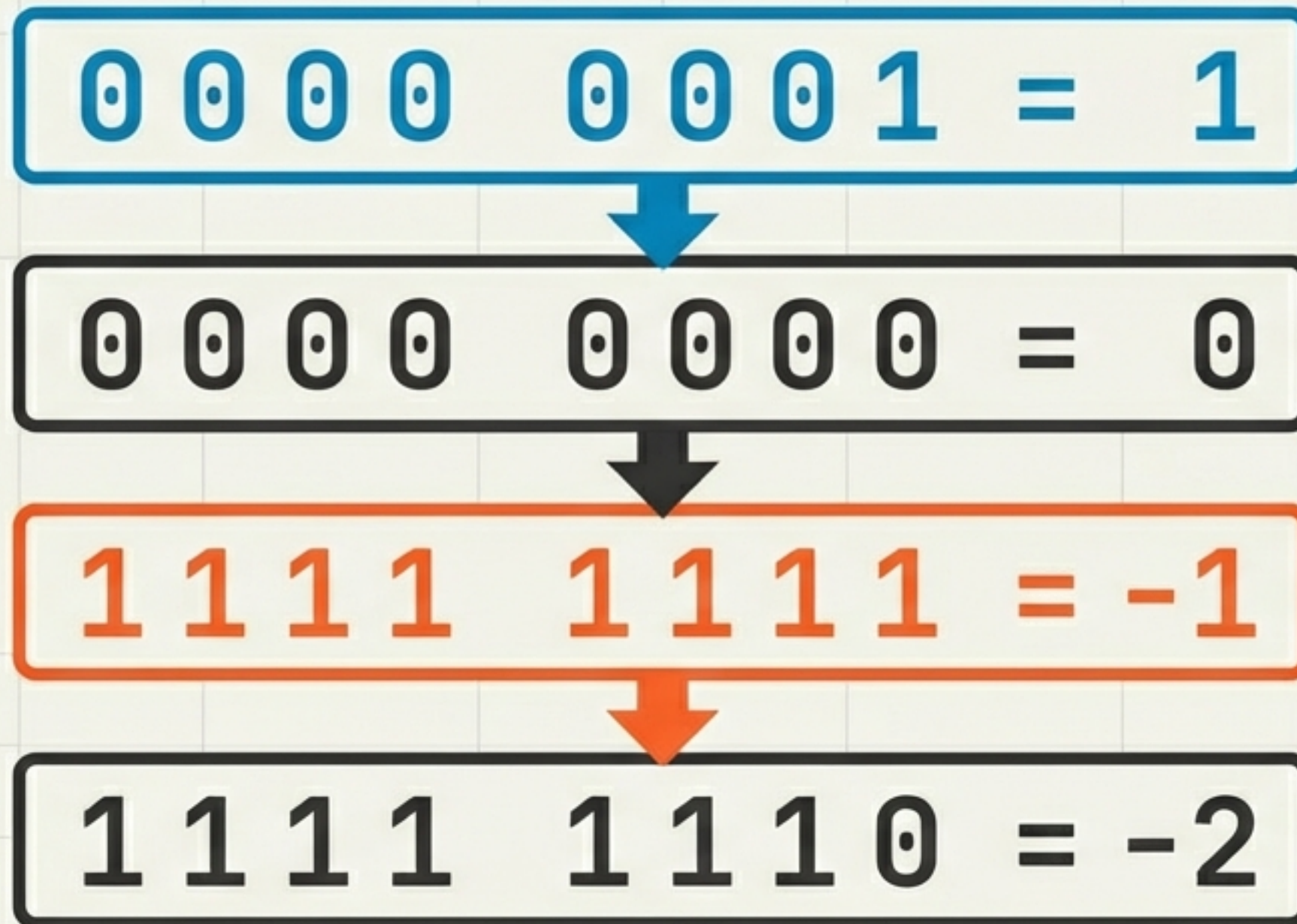
11111111111111111111111111111111

Critical Takeaway

- **Positive Numbers:** EDX must be zeroed.
- **Negative Numbers:** CDQ must fill EDX with sign bits.
- **Warning:** Missing this instruction leads to incorrect analysis of signed math.

Understanding Negatives: Two's Complement

Computers represent negative integers using the most significant bit as a sign indicator.



Calculation: The “Flip & Add” Method

Verifying the magnitude of a negative hex value (e.g., 0xFFFFFFFEB).

Convert to Binary

1111...1011

Invert Bits (Flip)

0000...0100

Add One

0000...0101

= 5

Original Value = -5

Knowledge Check: Variable Scope

Where are Global Variables stored?

→ In `.data` (initialized) or `.bss` (uninitialized).

What does '?' in `.data` signify?

→ Uninitialized data / Reserved space.

`mov eax, dword_402000`
implies...

→ A Global Variable
(Hardcoded address).

How are Local Variables referenced?

→ Offsets relative to EBP or ESP.

Knowledge Check: Mechanics

Where is the remainder stored after DIV?

→ The EDX Register.

What does CDQ do?

→ Sign-extends EAX into EDX (Sets up signed math).

Shortcut for Two's Complement magnitude?

→ Invert bits and Add 1.

Why rename variables?

→ To propagate meaning across the IDA database.

Summary & Best Practices



Mental Model

- Move from Micro to Macro.
- Focus on intent, not just instructions.



Identification

- Globals: Fixed (.data)
- Locals: Relative (Stack)



Technique

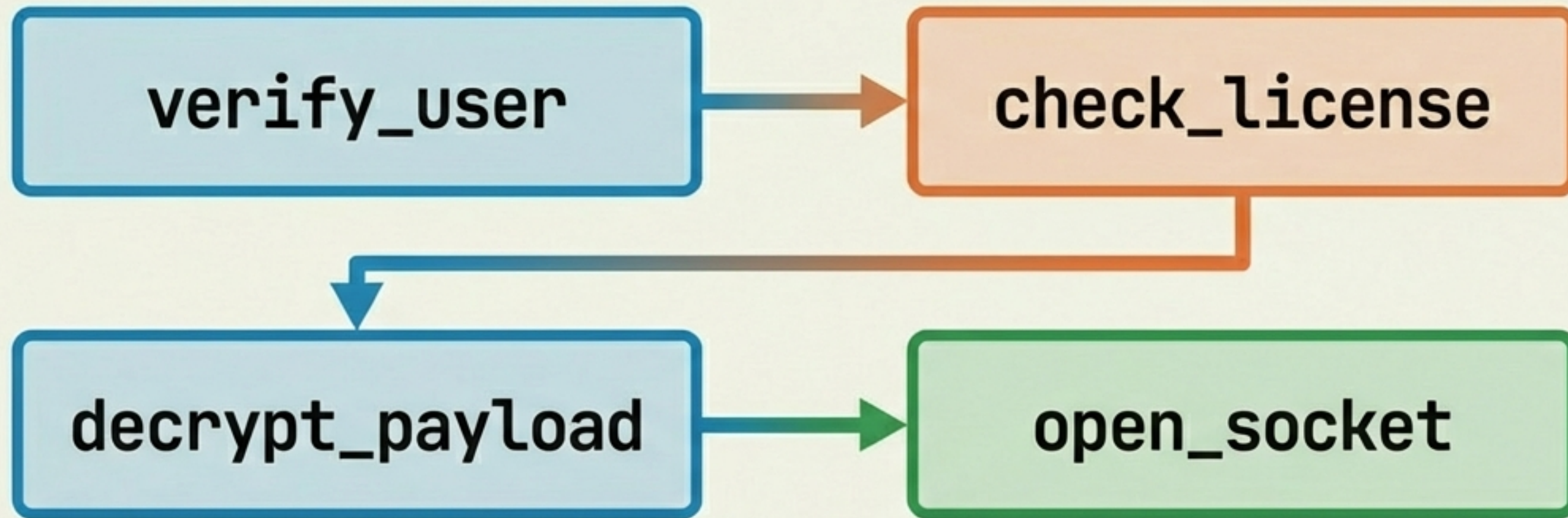
- Renaming is Analysis.
- Use Hotkey 'N' liberally.



Arithmetic

- Watch the EDX:EAX pair.
- Check for CDQ in signed math.

The Reconstruction



The assembly instructions are just the clues.
The logic is the story.